LE GUIDE COMPLET DU

BACKEND LARAVEL

Jargon, syntaxe, fonctionnement interne, problemes courants et questions d'entretien — explique simplement, du debutant a l'expert.

Prepare pour Djimi
Etudiant en Genie Informatique — IUT de Douala

Sommaire

1. Introduction : le monde du backend et de Laravel3
2. L'architecture MVC et le cycle de vie d'une requete5
3. Installation et structure d'un projet Laravel6
4. Le routage (Routing)8
5. Les controleurs (Controllers)11
6. Requetes, Validation et Reponses13
7. Eloquent ORM et base de donnees15
8. Blade : le moteur de templates21
9. Les Middlewares22
10. Service Container, Service Providers et Facades23
11. Authentification et Autorisation25
12. Events, Listeners et Observers26
13. Jobs, Queues et traitement asynchrone28
14. Notifications et Mailables29
15. Construire une API REST avec Laravel30
16. Stockage de fichiers, Cache et Sessions31
17. Les tests automatises32
18. Artisan CLI : commandes essentielles33
19. Design patterns utilises dans Laravel34
20. Glossaire complet du jargon backend Laravel (A a Z)35
21. Problemes courants rencontres en developpement Laravel (et leurs solutions)40
22. Questions d entretien frequentes (avec reponses)42
23. Conseils pratiques pour se preparer a un entretien Laravel / backend44
24. Pour aller plus loin45

Chapitre 1 — Introduction : le monde du backend et de Laravel

1.1 Qu'est-ce que le "backend" ?

> 💡 **Explication simple (comme a un bebe)**
> *Imagine un restaurant. La salle avec les tables, le menu, la deco, c'est ce que le client voit : c'est le FRONTEND (le site web, l'application que l'utilisateur touche avec sa souris ou son doigt). La cuisine, derriere une porte que le client ne voit jamais, ou le chef prepare les plats, gere le stock, verifie les commandes : c'est le BACKEND. Le backend, c'est tout ce qui se passe "en coulisses" : recevoir la commande du client, verifier qu'elle est valide, aller chercher les ingredients (les donnees) dans le frigo (la base de donnees), preparer le plat (traiter l'information) et le renvoyer sur la table (la reponse).*

Concretement, le backend d'une application web est le programme qui tourne sur un serveur (un ordinateur distant, allume 24h/24). Il recoit des requetes HTTP envoyees par le navigateur ou une application mobile, il execute une logique (calculs, verifications, regles metier), il lit ou ecrit des donnees dans une base de donnees, puis il renvoie une reponse (une page HTML, un fichier JSON, une image, etc.).

- Le FRONTEND = ce que l'utilisateur voit et manipule (boutons, formulaires, couleurs).
- Le BACKEND = la logique invisible qui traite les demandes et gere les donnees.
- La BASE DE DONNEES = l'endroit ou les informations sont rangees durablement (comme une immense armoire a fichiers).
- Le SERVEUR = l'ordinateur qui fait tourner le backend en permanence, accessible depuis Internet.

1.2 Qu'est-ce que Laravel ?

Laravel est un FRAMEWORK backend ecrit en langage PHP. Un framework, c'est une boite a outils et un squelette deja construits qui t'evitent de tout reecrire depuis zero a chaque projet.

> 💡 **Explication simple (comme a un bebe)**
> *Construire un site web sans framework, c'est comme construire une maison en fabriquant toi-meme chaque brique, chaque clou et chaque vis. Utiliser Laravel, c'est comme acheter une maison en kit : les fondations (gestion des requetes), la plomberie (connexion a la base de donnees), l'electricite (securite, authentification) sont deja prevues et testees. Toi, tu n'as plus qu'a amenager les pieces selon tes besoins (ta logique metier).*

Laravel a ete cree par Taylor Otwell en 2011. Il est aujourd'hui l'un des frameworks PHP les plus utilises au monde, tres apprecie pour sa syntaxe elegante ("expressive syntax"), sa documentation riche et son ecosysteme (Forge, Vapor, Nova, Sanctum, Livewire, Breeze, Jetstream, etc.).

Pourquoi choisir Laravel plutot que du PHP "pur" ?

- Securite integree : protection automatique contre les failles courantes (injection SQL, CSRF, XSS).
- Eloquent ORM : manipuler la base de donnees avec du PHP au lieu d'ecrire du SQL a la main.
- Systeme de routage clair et lisible.
- Outil en ligne de commande Artisan qui automatise les taches repetitives.
- Grande communaute : si tu as un probleme, quelqu'un l'a probablement deja resolu et partage la solution.
- Architecture qui impose de bonnes pratiques (organisation du code, separation des responsabilites).

1.3 Les prerequis indispensables

Avant Laravel, il faut connaitre un minimum de trois choses :

1. PHP : le langage de programmation sur lequel Laravel est construit (variables, fonctions, classes, objets, tableaux).
2. Composer : le gestionnaire de dependances de PHP. C'est lui qui telecharge Laravel et toutes les librairies (les "packages") dont ton projet a besoin.
3. SQL et les bases de donnees relationnelles : comprendre les notions de table, colonne, ligne, cle primaire, cle etrangere.

> 💡 **Explication simple (comme a un bebe)**
> *Composer, c'est comme une application de livraison de courses. Tu ecris ta liste de courses (le fichier composer.json), tu appuies sur "commander" (composer install), et Composer va chercher chaque ingredient (chaque librairie PHP) exactement dans la bonne version chez le bon fournisseur, puis les depose chez toi dans le dossier vendor/.*

```
# Verifier les versions installees
php -v
composer -V

# Installer Laravel via Composer
composer create-project laravel/laravel mon-projet

# Ou via l'installeur global de Laravel
laravel new mon-projet
```

Chapitre 2 — L'architecture MVC et le cycle de vie d'une requete

2.1 Le patron MVC (Model - View - Controller)

Laravel organise ton code selon le patron d'architecture MVC, qui separe trois responsabilites bien distinctes.

> 💡 **Explication simple (comme a un bebe)**
> *Reprenons le restaurant. Le CLIENT donne sa commande au SERVEUR (le Controleur). Le serveur ne cuisine pas lui-meme : il transmet la commande a la CUISINE (le Model, qui sait comment aller chercher les ingredients dans le stock, c'est-a-dire la base de donnees). Une fois le plat pret, il est dispose joliment dans une ASSIETTE (la Vue) avant d'etre servi au client. Le controleur fait le lien entre les trois : il recoit la demande, demande les donnees au model, choisit la vue, et renvoie le tout.*

- MODEL (Modele) : represente une "chose" de ton application (un Utilisateur, un Produit, une Commande) et sait comment lire/ecrire cette chose dans la base de donnees.
- VIEW (Vue) : ce que l'utilisateur voit au final ; en Laravel, ce sont les fichiers Blade (.blade.php).
- CONTROLLER (Controleur) : recoit la requete HTTP, orchestre le traitement (appelle le Model, applique la logique), puis renvoie une reponse (souvent une Vue ou du JSON).

2.2 Le cycle de vie d'une requete Laravel, etape par etape

> 💡 **Explication simple (comme a un bebe)**
> *Le cycle de vie d'une requete, c'est comme une lettre postee qui doit traverser plusieurs bureaux avant d'arriver au bon destinataire, puis la reponse refait le chemin inverse.*

1. Le navigateur envoie une requete HTTP (ex: GET /produits) qui arrive sur le serveur.
2. Le fichier public/index.php est le point d'entree UNIQUE de toute l'application Laravel : c'est la "porte d'entree".
3. index.php charge le fichier bootstrap/app.php qui cree le "Kernel" (le noyau HTTP), veritable chef d'orchestre de la requete.
4. Le Kernel fait passer la requete a travers une serie de MIDDLEWARES (des filtres successifs : verification de session, protection CSRF, etc.).
5. Le ROUTER (systeme de routage) regarde l'URL demandee et la methode HTTP, puis trouve la route correspondante dans routes/web.php ou routes/api.php.
6. La route pointe vers un CONTROLLER (ou une fonction anonyme, "closure").

7. Le controleur execute la logique : il peut interroger un MODEL Eloquent pour recuperer des donnees dans la base.
8. Le controleur retourne une reponse : une VUE Blade (HTML), une redirection, ou du JSON.
9. La reponse retraverse les middlewares (dans l'autre sens), puis est envoyee au navigateur.

Cette architecture en "pipeline" (une requete qui traverse une chaine d'etapes) est une notion fondamentale que l'on retrouve dans presque tous les frameworks backend modernes, pas seulement Laravel.

2.3 Le conteneur de services (apercu rapide)

Tout au long de ce cycle, Laravel s'appuie sur un "conteneur de services" (Service Container) qui fabrique et fournit automatiquement les objets dont chaque partie du code a besoin. Nous detaillerons ce mecanisme central au Chapitre 11, car c'est l'un des concepts les plus demandes en entretien.

Chapitre 3 — Installation et structure d'un projet Laravel

3.1 Installer Laravel

```
composer create-project laravel/laravel nom-du-projet
cd nom-du-projet
php artisan serve
# Le site est accessible sur http://127.0.0.1:8000
```

La commande "php artisan serve" demarre un petit serveur de developpement integre a PHP. En production, on utilise plutot un vrai serveur web comme Nginx ou Apache.

3.2 La structure des dossiers, expliquee dossier par dossier

> 💡 **Explication simple (comme a un bebe)**
> *Un projet Laravel, c'est comme une maison bien rangee : chaque piece a un role precis, et un developpeur experimente sait immediatement ou aller chercher quoi, meme dans un projet qu'il n'a jamais vu.*

Dossier / fichier	A quoi ca sert (explication simple)
app/	Le coeur de ton application : Models, Controllers, Middlewares, Providers, logique metier.

Dossier / fichier	A quoi ca sert (explication simple)
app/Models/	Toutes les classes representant tes "entites" (User, Product, Order...).
app/Http/Controllers/	Les controleurs qui recoivent les requetes et decident quoi faire.
app/Http/Middleware/	Les filtres qui interceptent les requetes avant/apres les controleurs.
bootstrap/	Fichiers de demarrage de l'application (dont bootstrap/app.php) et cache.
config/	Tous les fichiers de configuration (base de donnees, mail, cache, session...).
database/	Migrations (structure des tables), Seeders (donnees de test), Factories (generateurs de fausses donnees).
public/	Le SEUL dossier accessible directement par le navigateur ; contient index.php, le point d'entree.
resources/	Vues Blade, fichiers CSS/JS bruts, fichiers de traduction (lang).
routes/	Definition de toutes les URLs de l'application : web.php, api.php, console.php.
storage/	Fichiers generes par l'application : logs, fichiers uploades, cache de vues compilees.
tests/	Tests automatises (Feature et Unit).
vendor/	Toutes les librairies PHP installees par Composer (a ne JAMAIS modifier ni verser sur Git).
.env	Fichier de configuration secrete propre a chaque environnement (mots de passe, cles API) — ne doit jamais etre partage publiquement.
composer.json	La "carte d'identite" du projet : liste des dependances PHP et leurs versions.
artisan	Le script qui permet de lancer les commandes Artisan (php artisan ...).

3.3 Le fichier .env et la configuration

> 💡 **Explication simple (comme a un bebe)**
> *Le fichier .env, c'est comme le tableau de bord secret d'une voiture : il contient les reglages propres a CHAQUE environnement (ta machine perso, le serveur de test, le serveur de production) — par exemple les identifiants de la base de donnees. On ne le met jamais sur Git (on ajoute une ligne ".env" dans le fichier .gitignore) car il contient des informations sensibles.*

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:xxxxxxxxxxxxxxx
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ma_base
DB_USERNAME=root
DB_PASSWORD=
```

- APP_KEY : cle de chiffrement generee par "php artisan key:generate", indispensable au fonctionnement de Laravel.
- APP_DEBUG : si "true", affiche les erreurs detaillees (utile en developpement, DANGEREUX en production).
- Les valeurs de .env sont lues via la fonction env('CLE') dans les fichiers de config/, jamais directement ailleurs (bonne pratique : passer par config()).

Chapitre 4 — Le routage (Routing)

4.1 Qu'est-ce qu'une route ?

> 💡 **Explication simple (comme a un bebe)**
> *Une route, c'est comme une adresse postale associee a une action precise. Quand une lettre (une requete HTTP) arrive avec l'adresse "/produits" et le mode d'envoi "GET", Laravel regarde dans son "annuaire" (routes/web.php) et sait exactement quel bureau (quelle fonction ou quel controleur) doit traiter cette lettre.*

Les routes se definissent dans routes/web.php (pour les pages classiques, avec sessions et vues) et routes/api.php (pour les API, sans etat de session, generalement en JSON).

```
use Illuminate\Support\Facades\Route;

Route::get('/produits', function () {
    return 'Liste des produits';
```

```
});

Route::post('/produits', [ProductController::class, 'store']);
Route::put('/produits/{id}', [ProductController::class, 'update']);
Route::patch('/produits/{id}', [ProductController::class, 'update']);
Route::delete('/produits/{id}', [ProductController::class, 'destroy']);
```

4.2 Les verbes HTTP (methodes HTTP)

Verbe HTTP	A quoi il sert (analogie simple)
GET	LIRE une information (comme regarder une vitrine, sans rien changer).
POST	CREER une nouvelle information (comme deposer une nouvelle commande).
PUT	REMPLACER entierement une information existante (on redonne toute la fiche).
PATCH	MODIFIER partiellement une information existante (on corrige juste un champ).
DELETE	SUPPRIMER une information.

4.3 Parametres de route et Route Model Binding

Une route peut contenir des parametres dynamiques, entoures d'accolades.

```
Route::get('/produits/{id}', function ($id) {
    return 'Produit numero '.$id;
});

// Route Model Binding : Laravel va chercher automatiquement
// le Produit correspondant a l'id dans la base de donnees
Route::get('/produits/{product}', function (Product $product) {
    return $product->name;
});
```

> 💡 **Explication simple (comme a un bebe)**
> *Le Route Model Binding, c'est comme donner a un employe un numero de dossier : au lieu de lui donner juste le numero et de le laisser chercher le dossier lui-meme, Laravel va DIRECTEMENT chercher le dossier complet dans l'armoire et le lui poser sur le bureau. Cela evite d'ecrire manuellement Product::findOrFail($id) dans chaque controleur.*

4.4 Routes nommees

On peut donner un NOM a une route pour y faire reference facilement ailleurs dans le code, sans jamais ecrire l'URL en dur (ce qui evite de tout casser si l'URL change un jour).

```
Route::get('/profil', [ProfileController::class, 'show'])-
>name('profil.show');

// Utilisation ailleurs dans le code :
return redirect()->route('profil.show');
// Dans une vue Blade :
<a href="{{ route('profil.show') }}">Mon profil</a>
```

4.5 Groupes de routes et Middleware

On peut regrouper des routes qui partagent un prefixe, un middleware ou un espace de noms, pour eviter de repeter la meme configuration sur chaque ligne.

```
Route::middleware(['auth'])->prefix('admin')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'index'])-
>name('admin.dashboard');
    Route::get('/utilisateurs', [AdminController::class, 'users'])-
>name('admin.users');
});
```

> 💡 **Explication simple (comme a un bebe)**
> *Un groupe de routes avec middleware, c'est comme un badge d'acces pose a l'entree d'un couloir entier de bureaux : au lieu de verifier le badge devant chaque porte individuellement, on le verifie une seule fois a l'entree du couloir, et toutes les portes derriere sont protegees.*

4.6 Routes de ressources (Resource Routes)

Pour une ressource classique (CRUD complet : Create, Read, Update, Delete), Laravel propose un raccourci qui genere automatiquement 7 routes standards.

```
Route::resource('produits', ProductController::class);

// Equivaut a generer automatiquement :
// GET    /produits            -> index   (liste)
// GET    /produits/create     -> create  (formulaire de creation)
// POST   /produits            -> store   (enregistrer)
// GET    /produits/{id}       -> show    (afficher un element)
// GET    /produits/{id}/edit  -> edit    (formulaire de modification)
// PUT/PATCH /produits/{id}    -> update  (mettre a jour)
// DELETE /produits/{id}       -> destroy (supprimer)
```

CRUD est un acronyme fondamental du backend : Create (creer), Read (lire), Update (modifier), Delete (supprimer). Presque toutes les fonctionnalites d'une application se ramenent, d'une maniere ou d'une autre, a ces 4 operations.

Chapitre 5 — Les controleurs (Controllers)

5.1 Definition simple

> 💡 **Explication simple (comme a un bebe)**
> *Le controleur, c'est le "chef de salle" du restaurant. Il ne cuisine pas lui-meme, il ne sert pas directement les plats depuis la cuisine, mais il COORDONNE : il recoit la commande du client (la requete), il la transmet a la cuisine (le Model), il recupere le plat, verifie qu'il est bien presente, et le fait apporter au client (la reponse).*

Un controleur est une simple classe PHP qui regroupe des methodes, chacune correspondant a une action (afficher une liste, enregistrer un formulaire, etc.).

```
php artisan make:controller ProductController
php artisan make:controller ProductController --resource
php artisan make:controller ProductController --api
```

5.2 Un controleur "resource" complet

```
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function create()
    {
        return view('products.create');
    }

    public function store(Request $request)
    {
        $data = $request->validate([
            'name' => 'required|string|max:255',
            'price' => 'required|numeric|min:0',
        ]);
        Product::create($data);
        return redirect()->route('products.index');
    }

    public function show(Product $product)
```

```
    {
        return view('products.show', compact('product'));
    }

    public function edit(Product $product)
    {
        return view('products.edit', compact('product'));
    }

    public function update(Request $request, Product $product)
    {
        $data = $request->validate([
            'name' => 'required|string|max:255',
            'price' => 'required|numeric|min:0',
        ]);
        $product->update($data);
        return redirect()->route('products.index');
    }

    public function destroy(Product $product)
    {
        $product->delete();
        return redirect()->route('products.index');
    }
}
```

5.3 Single Action Controllers

Quand un controleur ne fait qu'UNE seule chose, on peut lui donner une seule methode magique appelee __invoke(), et le lier directement a une route sans preciser de nom de methode.

```
class GenererRapportController extends Controller
{
    public function __invoke()
    {
        // ...
    }
}

Route::get('/rapport', GenererRapportController::class);
```

5.4 L'injection de dependances dans les controleurs

Laravel "injecte" automatiquement les objets dont une methode a besoin, simplement en les indiquant en parametre (avec leur type). C'est le Service Container qui travaille en coulisses (voir Chapitre 11).

```
public function store(Request $request, MailerService $mailer)
{
    // $request et $mailer sont fournis automatiquement par Laravel
}
```

Chapitre 6 — Requetes, Validation et Reponses

6.1 L'objet Request

> 💡 **Explication simple (comme a un bebe)**
> *L'objet Request, c'est l'enveloppe complete de la lettre recue : elle contient non seulement le message (les donnees du formulaire), mais aussi l'expediteur (l'adresse IP), le type d'enveloppe (les en-tetes/headers), et les pieces jointes (les fichiers uploades).*

```
public function store(Request $request)
{
    $nom = $request->input('nom');
    $nom = $request->nom; // acces 'magique' equivalent
    $tous = $request->all();
    $fichier = $request->file('photo');
    $existe = $request->has('email');
    $methode = $request->method(); // GET, POST...
}
```

6.2 La validation

La validation, c'est le "controle qualite" avant d'accepter des donnees. On ne fait jamais confiance a ce qui vient de l'utilisateur : on verifie toujours le format, la presence, les limites.

```
$validated = $request->validate([
    'email' => 'required|email|unique:users,email',
    'age' => 'required|integer|min:18',
    'mot_de_passe' => 'required|min:8|confirmed',
]);
```

Si la validation echoue, Laravel redirige AUTOMATIQUEMENT l'utilisateur en arriere avec les messages d'erreur (accessibles dans la vue via la variable $errors), sans que le developpeur ait besoin d'ecrire ce mecanisme lui-meme.

6.3 Les Form Requests

Quand la validation devient longue, on la deplace dans une classe dediee appelee "Form Request", ce qui garde le controleur propre et court.

```
php artisan make:request StoreProductRequest
```

```
class StoreProductRequest extends FormRequest
{
    public function authorize()
    {
        return true; // ou une regle d'autorisation
    }

    public function rules()
    {
        return [
            'name' => 'required|string|max:255',
            'price' => 'required|numeric|min:0',
        ];
    }
}

// Dans le controleur :
public function store(StoreProductRequest $request)
{
    Product::create($request->validated());
}
```

6.4 Les types de reponses

Type de reponse	Exemple	Usage
Vue	return view('accueil');	Retourner une page HTML.
JSON	return response()->json(['ok' => true]);	Retourner des donnees pour une API.
Redirection	return redirect('/accueil');	Renvoyer l'utilisateur vers une autre page.
Redirection nommee	return redirect()->route('home');	Rediriger vers une route par son nom.
Telechargement	return response()->download($chemin);	Envoyer un fichier a telecharger.
Statut HTTP personnalise	return response('Non trouve', 404);	Preciser un code de statut (200, 404, 500...).

Un code de statut HTTP est un nombre a 3 chiffres qui resume le resultat d'une requete : 200 (succes), 201 (cree), 301/302 (redirection), 401 (non authentifie), 403 (interdit), 404 (introuvable), 419 (session/CSRF expire), 422 (donnees invalides), 500 (erreur serveur).

Chapitre 7 — Eloquent ORM et base de donnees

7.1 Qu'est-ce qu'un ORM ?

ORM signifie Object-Relational Mapping (mapping objet-relationnel). C'est un outil qui transforme les tables de la base de donnees en objets PHP manipulables, et inversement.

> 💡 **Explication simple (comme a un bebe)**
> *Sans ORM, pour parler a la base de donnees, il faut ecrire du SQL a la main, comme parler une langue etrangere phrase par phrase. Avec un ORM comme Eloquent, c'est comme avoir un traducteur personnel : tu ecris en PHP (ta langue), et le traducteur (Eloquent) se charge de convertir cela en SQL correct pour la base de donnees, puis retraduit le resultat en objets PHP faciles a utiliser.*

```
// Avec du SQL brut :
SELECT * FROM products WHERE price > 100;

// Avec Eloquent :
Product::where('price', '>', 100)->get();
```

7.2 Les migrations

Une migration est un fichier PHP qui decrit une modification de la structure de la base de donnees (creer une table, ajouter une colonne...). C'est comme un plan de construction versionne : chaque migration est une etape datee et ordonnee, ce qui permet a toute une equipe d'avoir exactement la meme structure de base de donnees, meme sur des machines differentes.

```
php artisan make:migration create_products_table
```

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description')->nullable();
        $table->decimal('price', 8, 2);
        $table->foreignId('category_id')->constrained();
        $table->timestamps();
    });
}

public function down()
{
    Schema::dropIfExists('products');
}
```

- up() decrit ce qu'il faut FAIRE (creer, ajouter).
- down() decrit comment ANNULER cette action (utile pour revenir en arriere).
- php artisan migrate execute toutes les migrations en attente.
- php artisan migrate:rollback annule la derniere serie de migrations.
- php artisan migrate:fresh supprime TOUTES les tables et relance toutes les migrations depuis zero (a utiliser uniquement en developpement).
- timestamps() ajoute automatiquement deux colonnes created_at et updated_at.

7.3 Les Models (modeles) Eloquent

```
php artisan make:model Product
php artisan make:model Product -m   // cree aussi la migration
php artisan make:model Product -mfc // migration + factory + controller
```

```
class Product extends Model
{
    protected $fillable = ['name', 'description', 'price', 'category_id'];

    protected $hidden = ['secret_code'];

    protected $casts = [
        'price' => 'decimal:2',
        'is_active' => 'boolean',
    ];
}
```

> 💡 **Explication simple (comme a un bebe)**
> *Un Model, c'est la carte d'identite d'un type d'objet dans ta base de donnees. Par convention, Laravel devine tout seul que le model Product correspond a la table products (nom au pluriel), avec une cle primaire id, sans que tu aies rien a configurer.*

FILLABLE : la liste blanche des colonnes qu'on autorise a remplir en masse (via create() ou update()). C'est une protection contre la faille de "mass assignment" (assignation de masse), ou un utilisateur malveillant pourrait injecter des champs non prevus (comme is_admin) dans un formulaire. GUARDED fait l'inverse : on liste ce qui est INTERDIT plutot que ce qui est autorise.

7.4 Operations CRUD avec Eloquent

```
// CREATE
Product::create(['name' => 'Chaise', 'price' => 49.99]);

// READ
```

```
Product::all();
Product::find(1);
Product::findOrFail(1);
Product::where('price', '>', 20)->get();
Product::where('price', '>', 20)->first();

// UPDATE
$product = Product::find(1);
$product->price = 59.99;
$product->save();
// ou
$product->update(['price' => 59.99]);

// DELETE
$product->delete();
Product::destroy(1);
```

7.5 Les relations entre modeles

> 💡 **Explication simple (comme a un bebe)**
> *Les relations Eloquent, ce sont les liens de famille entre tes tables : qui est le parent de qui, qui possede quoi. Comprendre ces relations est absolument central en entretien Laravel.*

Relation	Exemple concret	Analogie
hasOne	Un Utilisateur a UN Passeport	Une personne a une seule carte d'identite.
belongsTo	Un Passeport appartient a UN Utilisateur	La carte d'identite appartient a une personne precise.
hasMany	Un Utilisateur a PLUSIEURS Commandes	Un client peut avoir plusieurs factures dans son dossier.
belongsToMany	Un Etudiant a PLUSIEURS Cours, et un Cours a PLUSIEURS Etudiants	Comme un carnet de rendez-vous partage entre plusieurs personnes et plusieurs salles.
hasManyThrough	Un Pays a PLUSIEURS Commentaires via ses Utilisateurs	Une relation "a travers" un intermediaire.
Polymorphic (morphTo/morphMany)	Un Commentaire peut appartenir a un Article OU a une Video	Une meme etiquette qui peut se coller sur des objets de types differents.

```
class User extends Model
{
    public function orders()
```

```
    {
        return $this->hasMany(Order::class);
    }
}

class Order extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}

// Utilisation :
$user->orders;         // Collection de commandes
$order->user;          // L'utilisateur proprietaire
$user->orders()->create(['total' => 100]);
```

Pour une relation belongsToMany, Laravel utilise une TABLE PIVOT (une table intermediaire) qui contient simplement les deux cles etrangeres, par exemple course_user, avec course_id et user_id.

```
class Student extends Model
{
    public function courses()
    {
        return $this->belongsToMany(Course::class);
    }
}

$student->courses()->attach($courseId);
$student->courses()->detach($courseId);
$student->courses()->sync([1, 2, 3]);
```

7.6 Query Builder vs Eloquent

Le Query Builder est une couche plus "brute" qui construit des requetes SQL sans passer par des objets Model. Eloquent est construit AU-DESSUS du Query Builder.

```
// Query Builder (retourne des objets stdClass generiques)
DB::table('products')->where('price', '>', 100)->get();

// Eloquent (retourne des instances du Model Product, avec toutes ses
methodes)
Product::where('price', '>', 100)->get();
```

7.7 Eager Loading et le probleme N+1

Le probleme N+1 est L'UNE DES QUESTIONS D'ENTRETIEN LES PLUS FREQUENTES en backend. Il se produit quand on charge une liste de N elements, puis qu'on effectue une requete SUPPLEMENTAIRE pour chaque element afin de recuperer une relation : cela fait 1 requete initiale + N requetes = N+1 requetes, au lieu de seulement 2.

> 💡 **Explication simple (comme a un bebe)**
> *Imagine que tu demandes la liste des 50 employes d'une entreprise (1 requete), puis que, pour connaitre le departement de CHAQUE employe, tu retournes demander individuellement 50 fois (50 requetes) au lieu de demander une seule fois "donne-moi les employes ET leurs departements en meme temps". C'est extrememement lent et ca fait exploser le nombre d'appels a la base de donnees.*

```
// MAUVAIS : probleme N+1 (1 requete + N requetes dans la boucle)
$orders = Order::all();
foreach ($orders as $order) {
    echo $order->user->name; // 1 requete SQL par commande !
}

// BON : Eager Loading avec with() (seulement 2 requetes au total)
$orders = Order::with('user')->get();
foreach ($orders as $order) {
    echo $order->user->name; // deja charge, aucune requete supplementaire
}
```

On peut detecter ce probleme en developpement avec l'outil "Laravel Debugbar" ou en interdisant le "lazy loading" via Model::preventLazyLoading() dans un environnement de developpement.

7.8 Seeders et Factories

Un SEEDER remplit la base de donnees avec des donnees de depart (ex: des categories par defaut). Une FACTORY genere de FAUSSES donnees realistes en grande quantite, tres utile pour tester l'application.

```
php artisan make:seeder ProductSeeder
php artisan make:factory ProductFactory --model=Product
```

```
// database/factories/ProductFactory.php
public function definition()
{
    return [
        'name' => fake()->words(3, true),
        'price' => fake()->randomFloat(2, 5, 500),
    ];
}
```

```
// Utilisation :
Product::factory()->count(50)->create();
```

7.9 Les transactions

Une TRANSACTION garantit qu'un groupe d'operations sur la base de donnees s'execute "tout ou rien" : si une etape echoue au milieu, TOUT est annule (rollback), pour eviter des donnees incoherentes.

> 💡 **Explication simple (comme a un bebe)**
> *Un virement bancaire entre deux comptes doit debiter le compte A ET crediter le compte B. Si le systeme plante juste apres avoir debite A mais avant de crediter B, l'argent disparaitrait ! Une transaction garantit que les deux operations reussissent ensemble, ou qu'aucune des deux n'a lieu.*

```
DB::transaction(function () {
    $compteA->decrement('solde', 100);
    $compteB->increment('solde', 100);
});
```

7.10 Accesseurs, mutateurs et scopes

- Accesseur (Accessor) : transforme une valeur AU MOMENT DE LA LIRE depuis la base.
- Mutateur (Mutator) : transforme une valeur AU MOMENT DE L'ENREGISTRER dans la base.
- Scope local : une methode reutilisable pour filtrer une requete Eloquent, ex: Product::active()->get().
- Soft Delete (suppression douce) : au lieu de supprimer vraiment une ligne, on marque juste une colonne deleted_at, ce qui permet de la restaurer plus tard.

```
// Accesseur (Laravel 9+)
protected function name(): Attribute
{
    return Attribute::make(
        get: fn ($value) => ucfirst($value),
    );
}

// Scope local
public function scopeActive($query)
{
    return $query->where('is_active', true);
}
// Utilisation : Product::active()->get();
```

```
// Soft Delete
class Product extends Model
{
    use SoftDeletes;
}
```

Chapitre 8 — Blade : le moteur de templates

8.1 Qu'est-ce que Blade ?

> 💡 **Explication simple (comme a un bebe)**
> *Blade, c'est comme une lettre type avec des espaces reserves ("Cher [NOM], votre commande [NUMERO] est prete"). Le fichier .blade.php contient du HTML normal, mais avec des zones speciales que Laravel remplit automatiquement avec les vraies donnees avant d'envoyer la page au navigateur.*

```
{{-- resources/views/produits/show.blade.php --}}
<h1>{{ $product->name }}</h1>
<p>Prix : {{ $product->price }} FCFA</p>

@if ($product->is_active)
    <span>Disponible</span>
@else
    <span>Rupture de stock</span>
@endif

<ul>
@foreach ($product->reviews as $review)
    <li>{{ $review->comment }}</li>
@endforeach
</ul>
```

La double accolade {{ $variable }} affiche une valeur en l'echappant AUTOMATIQUEMENT (elle convertit les caracteres speciaux HTML), ce qui protege contre les attaques XSS (Cross-Site Scripting), ou un utilisateur malveillant tenterait d'injecter du code HTML/JavaScript dans un champ de formulaire.

8.2 Layouts et composants

Pour eviter de repeter le meme header/footer sur chaque page, Blade permet de definir un LAYOUT (gabarit principal) que chaque page vient "completer".

```
{{-- resources/views/layouts/app.blade.php --}}
<html>
<body>
```

```
    @yield('content')
</body>
</html>

{{-- resources/views/accueil.blade.php --}}
@extends('layouts.app')
@section('content')
    <h1>Bienvenue</h1>
@endsection
```

Les COMPOSANTS Blade (@component ou <x-nom-composant>) permettent de reutiliser des morceaux d'interface (un bouton, une carte produit) avec leurs propres donnees, un peu comme des "briques Lego" d'interface.

Chapitre 9 — Les Middlewares

9.1 Definition simple

> 💡 **Explication simple (comme a un bebe)**
> *Un middleware, c'est un AGENT DE SECURITE poste a l'entree d'un batiment. Chaque personne (chaque requete) qui veut entrer doit passer devant lui. L'agent peut : la laisser passer telle quelle, la bloquer completement (par exemple si elle n'a pas de badge), ou meme la rediriger ailleurs. Plusieurs agents peuvent se succeder les uns apres les autres avant d'atteindre le controleur.*

Techniquement, un middleware est une classe qui intercepte une requete AVANT (ou une reponse APRES) qu'elle n'atteigne le controleur. Exemples integres : auth (verifie que l'utilisateur est connecte), throttle (limite le nombre de requetes), verified (verifie que l'email est confirme).

```
php artisan make:middleware EstAdmin
```

```
class EstAdmin
{
    public function handle(Request $request, Closure $next)
    {
        if (! $request->user()?->is_admin) {
            abort(403);
        }
        return $next($request); // laisse la requete continuer son chemin
    }
}
```

La ligne $next($request) est essentielle : c'est elle qui "laisse passer" la requete vers l'etape suivante du pipeline. Si elle n'est jamais appelee, la requete s'arrete la, et le controleur n'est jamais atteint.

On enregistre ensuite le middleware (dans bootstrap/app.php pour Laravel 11+, ou app/Http/Kernel.php pour les versions anterieures) puis on l'applique a une route, par exemple avec ->middleware(admin) ajoute a la fin d'une declaration de route.

Chapitre 10 — Service Container, Service Providers et Facades

10.1 L'injection de dependances (Dependency Injection)

> 💡 **Explication simple (comme a un bebe)**
> *Imagine un robot cuisinier qui a besoin d'un couteau pour travailler. Au lieu de FABRIQUER lui-meme son couteau a chaque fois (ce qui le rendrait rigide et difficile a reparer), on le lui FOURNIT ("injecte") depuis l'exterieur. Si demain on veut lui donner un couteau different (plus tranchant, ou factice pour un test), il suffit de changer ce qu'on lui fournit, sans toucher au robot lui-meme.*

L'injection de dependances (DI) est un principe : une classe ne doit pas creer elle-meme les objets dont elle depend, elle doit les recevoir de l'exterieur (souvent via son constructeur ou les parametres d'une methode).

10.2 Le Service Container (conteneur IoC)

IoC signifie "Inversion of Control" (inversion de controle) : au lieu que ton code controle la creation de ses dependances, c'est le FRAMEWORK qui en prend le controle et te les fournit au bon moment. Le Service Container de Laravel est la "boite" centrale qui sait comment CONSTRUIRE et FOURNIR (injecter) automatiquement n'importe quel objet dont une classe a besoin.

> 💡 **Explication simple (comme a un bebe)**
> *Le Service Container, c'est comme un majordome ultra-efficace dans un grand hotel : des que quelqu'un a besoin de quelque chose (une serviette, une cle, un service), il sait exactement ou l'obtenir et le lui apporte instantanement, sans que la personne ait besoin de savoir comment ni ou c'est fabrique.*

```
// Liaison ('binding') dans un Service Provider :
$this->app->bind(PaiementInterface::class, StripePaiement::class);

// Singleton : la MEME instance est reutilisee partout dans l'application
$this->app->singleton(CacheManager::class, function ($app) {
    return new CacheManager();
});

// Des qu'une classe demande un PaiementInterface dans son constructeur,
```

```
// Laravel fournit automatiquement une instance de StripePaiement.
class CommandeController extends Controller
{
    public function __construct(protected PaiementInterface $paiement) {}
}
```

- bind() : cree une NOUVELLE instance a chaque fois qu'on la demande.
- singleton() : cree UNE SEULE instance, reutilisee pour toute la duree de la requete.
- Interface (Contrat) : une "promesse" de methodes qu'une classe doit implementer, sans dire COMMENT. Cela permet de changer facilement d'implementation (ex: passer de Stripe a PayPal) sans modifier le code qui utilise l'interface.

10.3 Les Service Providers

Un Service Provider est l'endroit CENTRAL ou l'on configure et "enregistre" les liaisons du Service Container, ainsi que tout ce qui doit demarrer avec l'application (routes, evenements, vues partagees...). Chaque fonctionnalite majeure de Laravel (base de donnees, routage, sessions...) est en realite demarree par son propre Service Provider.

```
class PaiementServiceProvider extends ServiceProvider
{
    public function register()
    {
        $this->app->bind(PaiementInterface::class, StripePaiement::class);
    }

    public function boot()
    {
        // Code execute une fois que TOUS les providers sont enregistres
    }
}
```

10.4 Les Facades

> 💡 **Explication simple (comme a un bebe)**
> *Une Facade, c'est comme le tableau de bord d'une voiture : tu tournes la cle (Auth::user()), tu appuies sur l'accelerateur (Cache::put(...)), sans jamais voir le moteur complexe qui tourne en dessous. C'est une facade (une "facade" au sens architectural) qui donne un ACCES SIMPLE et STATIQUE a des objets en realite geres par le Service Container.*

```
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Auth;

Cache::put('cle', 'valeur', 60);
$utilisateurConnecte = Auth::user();
```

Une question tres frequente en entretien : "Facade vs Injection de dependances ?" Les deux appellent finalement le meme objet sous-jacent depuis le Service Container. La Facade est plus courte a ecrire et tres lisible, mais l'injection de dependances explicite (via le constructeur) rend les dependances d'une classe plus visibles et facilite les tests unitaires (on peut plus facilement remplacer une dependance par une fausse version, un "mock").

Chapitre 11 — Authentification et Autorisation

11.1 La difference fondamentale : Authentification vs Autorisation

> 💡 **Explication simple (comme a un bebe)**
> *L AUTHENTIFICATION repond a la question QUI ES-TU ? (verifier l identite, comme montrer sa carte d identite a l entree). L AUTORISATION repond a la question AS-TU LE DROIT DE FAIRE CA ? (verifier les permissions, comme verifier si ta carte d identite te donne acces a la salle VIP). On peut etre authentifie (connu du systeme) sans etre autorise a faire une action precise.*

11.2 Guards et Providers

Un GUARD (garde) definit COMMENT un utilisateur est authentifie pour chaque requete (par exemple via une session de navigateur, ou via un jeton API). Un PROVIDER definit OU aller chercher les utilisateurs (generalement la table users via Eloquent).

```
Auth::check();          // true si un utilisateur est connecte
Auth::user();           // l utilisateur actuellement connecte
Auth::id();             // son identifiant
Auth::attempt(['email' => $email, 'password' => $mdp]);
Auth::logout();
```

11.3 Gates et Policies

Une GATE (porte) est une regle d autorisation SIMPLE, souvent basee sur une closure, utile pour des actions generales. Une POLICY (politique) est une classe dediee qui regroupe TOUTES les regles d autorisation pour UN Model precis (par exemple, qui peut modifier ou supprimer un Article).

```
php artisan make:policy ArticlePolicy --model=Article
```

```
class ArticlePolicy
{
    public function update(User $user, Article $article)
```

```
    {
        return $user->id === $article->user_id;
    }
}

// Utilisation dans un controleur :
public function update(Article $article)
{
    $this->authorize('update', $article);
    // ...
}

// Utilisation dans une vue Blade :
// @can('update', $article) ... @endcan
```

11.4 Panorama des outils d authentification Laravel

Outil	Cas d usage
Laravel Breeze	Kit minimaliste pret a l emploi pour inscription/connexion (Blade ou Vue/React via Inertia).
Laravel Jetstream	Version plus complete de Breeze : equipes, authentification a deux facteurs, gestion de session.
Laravel Sanctum	Authentification legere par jetons (tokens) pour API et applications SPA (Single Page Application).
Laravel Passport	Implementation complete du protocole OAuth2, pour des besoins d authentification API avances (applications tierces).

> 💡 **Explication simple (comme a un bebe)**
> *Sanctum, c est un badge d acces simple pour ton propre immeuble (ton API et ton frontend). Passport, c est un systeme de badges beaucoup plus complexe permettant a des ENTREPRISES EXTERIEURES d autoriser leurs propres employes a acceder a certaines parties de ton immeuble, avec des niveaux de permission tres fins (les scopes OAuth2).*

Chapitre 12 — Events, Listeners et Observers

12.1 Events et Listeners

> 💡 **Explication simple (comme a un bebe)**

> *Un EVENEMENT (Event), c est comme une alarme incendie qu on declenche ("Une commande vient d etre passee !"). Un ECOUTEUR (Listener), c est chaque personne qui, en entendant l alarme, effectue SA propre action independamment des autres : l un envoie un email, l autre met a jour un stock, un troisieme notifie l equipe logistique. Celui qui declenche l alarme n a pas besoin de savoir qui va reagir ni comment.*

Ce mecanisme permet de DECOUPLER le code : le controleur qui cree la commande n a pas besoin de connaitre tous les traitements annexes a declencher, il se contente de "signaler" l evenement.

```
php artisan make:event CommandePassee
php artisan make:listener EnvoyerEmailConfirmation --event=CommandePassee
```

```
// Declencher l evenement :
event(new CommandePassee($commande));

// Le Listener reagit :
class EnvoyerEmailConfirmation
{
    public function handle(CommandePassee $event)
    {
        Mail::to($event->commande->user)->send(new
ConfirmationCommande($event->commande));
    }
}
```

12.2 Les Observers

Un OBSERVER (observateur) regroupe la logique a executer automatiquement a des moments cles du cycle de vie d un Model : quand il est cree (created), mis a jour (updated), supprime (deleted), etc. C est une alternative aux evenements, plus specifique aux Models.

```
php artisan make:observer ProductObserver --model=Product
```

```
class ProductObserver
{
    public function created(Product $product)
    {
        Log::info('Nouveau produit cree : '.$product->name);
    }
}
```

Chapitre 13 — Jobs, Queues et traitement asynchrone

13.1 Pourquoi des files d attente (Queues) ?

> 💡 **Explication simple (comme a un bebe)**
> *Imagine un restaurant ou le serveur devrait rester plante devant le client jusqu a ce que le plat soit entierement cuisine (parfois 30 minutes) avant de pouvoir aller servir la table suivante. Ce serait absurde ! A la place, le serveur donne la commande a la cuisine et va IMMEDIATEMENT servir la table suivante ; le plat sera apporte des qu il sera pret. Les Queues (files d attente) font exactement ca pour ton application : au lieu de faire attendre l utilisateur pendant qu une tache LONGUE s execute (envoyer un email, generer un gros PDF, appeler une API externe lente), on met cette tache DANS UNE FILE D ATTENTE et on repond immediatement a l utilisateur. La tache sera traitee en arriere-plan par un "worker" (un ouvrier).*

Un JOB (tache) est une classe PHP qui represente UNE tache a executer, potentiellement en arriere-plan.

```
php artisan make:job GenererRapportPDF
```

```
class GenererRapportPDF implements ShouldQueue
{
    use Queueable;

    public function __construct(protected Commande $commande) {}

    public function handle()
    {
        // logique longue : generation du PDF, envoi, etc.
    }
}

// Envoyer le job dans la file d attente :
GenererRapportPDF::dispatch($commande);
```

Pour que les jobs soient reellement traites, il faut lancer un WORKER, un processus qui tourne en continu et "consomme" les jobs de la file :

```
php artisan queue:work
```

Laravel supporte plusieurs pilotes ("drivers") de file d attente : database, redis, sqs (Amazon), ou sync (execution immediate, utile en developpement/tests).

13.2 Broadcasting (diffusion temps reel)

Le BROADCASTING permet d envoyer des evenements EN TEMPS REEL vers le navigateur (par exemple, une notification qui apparait instantanement sans recharger la page), generalement via WebSockets et des outils comme Laravel Reverb, Pusher ou Soketi.

Chapitre 14 — Notifications et Mailables

14.1 Les Mailables

Un MAILABLE est une classe qui represente UN email : son contenu, sa mise en forme, ses pieces jointes. Cela evite d ecrire la logique d envoi d email directement dans les controleurs.

```
php artisan make:mail ConfirmationCommande
```

```
class ConfirmationCommande extends Mailable
{
    public function __construct(public Commande $commande) {}

    public function build()
    {
        return $this->view('emails.confirmation')
                    ->subject('Votre commande est confirmee');
    }
}

// Envoi :
Mail::to($user)->send(new ConfirmationCommande($commande));
```

14.2 Les Notifications

Le systeme de NOTIFICATIONS de Laravel va plus loin que les Mailables : une meme notification peut etre envoyee simultanement par plusieurs CANAUX (mail, SMS, Slack, base de donnees pour affichage dans l interface) sans dupliquer le code.

```
php artisan make:notification NouvelleCommande
```

```
class NouvelleCommande extends Notification
{
    public function via($notifiable)
    {
        return ['mail', 'database'];
    }
```

```
    public function toMail($notifiable)
    {
        return (new MailMessage)->line('Une nouvelle commande a ete
passee.');
    }
}

$user->notify(new NouvelleCommande($commande));
```

Chapitre 15 — Construire une API REST avec Laravel

15.1 Qu est-ce qu une API REST ?

API signifie Application Programming Interface : une interface qui permet a deux logiciels de communiquer entre eux (par exemple, une application mobile qui parle a ton backend Laravel). REST (Representational State Transfer) est un STYLE d architecture qui impose quelques regles simples : utiliser les verbes HTTP de maniere coherente (GET, POST, PUT, DELETE), organiser les URLs autour de RESSOURCES (/produits, /produits/5), et rester STATELESS (sans etat : chaque requete doit contenir toutes les informations necessaires, le serveur ne "se souvient" pas de la requete precedente).

> 💡 **Explication simple (comme a un bebe)**
> *Une API, c est comme le menu et le comptoir d un fast-food self-service : tu ne rentres pas en cuisine toi-meme, tu passes ta commande selon un format precis et standardise au comptoir (l endpoint), et tu recois ta commande dans un format previsible.*

15.2 Les API Resources

Une API RESOURCE est une classe qui transforme un Model Eloquent en un tableau JSON precis et controle, en cachant les champs sensibles et en ajoutant si besoin des donnees calculees.

```
php artisan make:resource ProductResource
```

```
class ProductResource extends JsonResource
{
    public function toArray($request)
    {
        return [
            'id' => $this->id,
            'nom' => $this->name,
            'prix_formate' => number_format($this->price, 2).' FCFA',
        ];
    }
```

```
}

// Dans le controleur :
return ProductResource::collection(Product::all());
return new ProductResource($product);
```

15.3 Le rate limiting (limitation de debit)

Le RATE LIMITING protege une API contre les abus en limitant le nombre de requetes qu un meme utilisateur (ou une meme adresse IP) peut effectuer sur une periode donnee, via le middleware throttle.

```
Route::middleware('throttle:60,1')->group(function () {
    // maximum 60 requetes par minute
});
```

Chapitre 16 — Stockage de fichiers, Cache et Sessions

16.1 Le systeme de fichiers (Storage)

Laravel fournit une abstraction unifiee pour manipuler des fichiers, que ce soit sur le disque local, ou sur un service distant comme Amazon S3, sans changer le code applicatif (seule la configuration du "disque" change).

```
Storage::put('avatars/photo.jpg', $contenu);
Storage::disk('s3')->put('rapports/rapport.pdf', $contenu);
$url = Storage::url('avatars/photo.jpg');
```

16.2 Le Cache

> 💡 **Explication simple (comme a un bebe)**
> *Le CACHE, c est comme un pense-bete colle sur le frigo : au lieu de refaire un calcul long ou une requete couteuse a chaque fois, on GARDE le resultat sous la main pendant un moment, pour le reutiliser instantanement la prochaine fois qu on en a besoin.*

```
Cache::put('produits_populaires', $produits, 3600); // stocke 1h
$produits = Cache::remember('produits_populaires', 3600, function () {
    return Product::orderBy('ventes', 'desc')->take(10)->get();
});
```

16.3 Les Sessions

Une SESSION permet de conserver des informations propres a UN utilisateur ENTRE plusieurs requetes (par exemple, rester connecte en naviguant de page en page), generalement via un cookie contenant un identifiant de session.

```
session(['panier' => $items]);
$panier = session('panier');
```

Chapitre 17 — Les tests automatises

17.1 Pourquoi tester ?

> 💡 **Explication simple (comme a un bebe)**
> *Un test automatise, c est comme un controle qualite robotise en usine : au lieu de verifier a la main chaque piece produite, une machine verifie automatiquement, a chaque nouvelle production, que tout fonctionne encore comme prevu. Cela permet de detecter immediatement si une modification a "casse" quelque chose ailleurs.*

17.2 Tests Unitaires vs Tests Fonctionnels (Feature)

Type de test	Ce qu il verifie
Unit Test (test unitaire)	Une SEULE unite de code isolee (une fonction, une methode), sans base de donnees ni framework complet.
Feature Test (test fonctionnel)	Un comportement complet de l application, souvent via une requete HTTP simulee (ex : verifier qu un formulaire d inscription fonctionne de bout en bout).

```
php artisan make:test ProductTest
php artisan make:test ProductUnitTest --unit
```

```
public function test_un_utilisateur_peut_voir_la_liste_des_produits()
{
    Product::factory()->count(3)->create();

    $response = $this->get('/produits');

    $response->assertStatus(200);
    $response->assertSeeText('Liste des produits');
}
```

Laravel utilise PHPUnit par defaut, mais l ecosysteme propose aussi Pest, une syntaxe plus courte et plus lisible, de plus en plus populaire.

Chapitre 18 — Artisan CLI : commandes essentielles

Artisan est l outil en ligne de commande fourni avec Laravel. C est un GENERATEUR et un ASSISTANT qui automatise la creation de fichiers et l execution de taches courantes.

Commande	A quoi elle sert
php artisan serve	Demarre un serveur de developpement local.
php artisan make:model Nom	Cree un nouveau Model.
php artisan make:controller Nom	Cree un nouveau Controller.
php artisan make:migration nom	Cree une nouvelle migration.
php artisan make:seeder Nom	Cree un nouveau Seeder.
php artisan make:factory Nom	Cree une nouvelle Factory.
php artisan make:middleware Nom	Cree un nouveau Middleware.
php artisan make:request Nom	Cree un Form Request de validation.
php artisan make:resource Nom	Cree une API Resource.
php artisan make:event / make:listener	Cree un Event ou un Listener.
php artisan make:job Nom	Cree un nouveau Job (file d attente).
php artisan make:policy Nom	Cree une Policy d autorisation.
php artisan migrate	Execute les migrations en attente.
php artisan migrate:rollback	Annule la derniere serie de migrations.
php artisan migrate:fresh --seed	Recree toute la base et relance les seeders.
php artisan db:seed	Execute les seeders.
php artisan tinker	Ouvre une console interactive PHP avec l application chargee.
php artisan route:list	Affiche toutes les routes de l application.
php artisan queue:work	Demarre un worker pour traiter les jobs en file d attente.
php artisan config:cache	Met en cache la configuration pour accelerer l application (production).

Commande	A quoi elle sert
php artisan cache:clear	Vide le cache de l application.
php artisan key:generate	Genere la cle de chiffrement APP_KEY.
php artisan test	Execute la suite de tests.

> 💡 **Explication simple (comme a un bebe)**
> *Tinker, c est un peu comme une salle de bricolage ou tu peux tester en direct des morceaux de ton application (creer un utilisateur, tester une relation Eloquent) sans avoir a passer par une page web complete.*

Chapitre 19 — Design patterns utilises dans Laravel

Un DESIGN PATTERN (patron de conception) est une solution EPROUVEE et REUTILISABLE a un probleme de conception logicielle courant. Laravel en utilise plusieurs en interne, et les connaitre est tres valorise en entretien.

Design Pattern	Ou on le retrouve dans Laravel
Singleton	Le Service Container garantit une seule instance pour certains services enregistres via singleton().
Facade	Les Facades (Cache::, Auth::, Route::) donnent un acces simplifie a des objets complexes.
Observer	Le systeme d Observers sur les Models (created, updated, deleted).
Repository	Pattern populaire (non impose par Laravel) qui isole la logique d acces aux donnees derriere une interface, pour decoupler les controleurs d Eloquent.
Factory	Les Model Factories generent des objets de test ; le pattern Factory design plus general se retrouve aussi dans la creation d objets complexes.
Strategy	Les differents "drivers" interchangeables (cache, session, queue, filesystem) suivent ce principe : une meme interface, plusieurs implementations.

Design Pattern	Ou on le retrouve dans Laravel
Chain of Responsibility	Le pipeline des Middlewares : chaque middleware traite la requete puis la transmet au suivant.
Builder	Le Query Builder construit une requete SQL etape par etape via des methodes chainees (->where()->orderBy()->get()).

Chapitre 20 — Glossaire complet du jargon backend Laravel (A a Z)

Ce glossaire reprend, sous forme de definitions courtes et simples, les termes les plus importants a maitriser. Beaucoup ont deja ete expliques en detail dans les chapitres precedents : ce glossaire sert de reference rapide et de fiche de revision avant un entretien.

A

- Accessor (Accesseur) : methode qui transforme une valeur au moment ou on la LIT depuis un Model.
- API (Application Programming Interface) : interface qui permet a deux logiciels de communiquer.
- Artisan : outil en ligne de commande de Laravel pour generer du code et automatiser des taches.
- Authentification : verifier QUI est l utilisateur.
- Autorisation : verifier ce que l utilisateur A LE DROIT de faire.
- Autoloading (chargement automatique) : mecanisme PHP (via Composer/PSR-4) qui charge automatiquement les classes sans require manuel.

B

- Blade : le moteur de templates (gabarits de vues) de Laravel.
- Binding : action d associer une interface (ou un nom) a une implementation concrete dans le Service Container.
- Broadcasting : diffusion d evenements en temps reel vers le navigateur (WebSockets).

C

- Cache : stockage temporaire de resultats pour eviter de refaire un calcul ou une requete couteuse.
- Cast (Casting) : conversion automatique du type d un attribut de Model (ex : string vers boolean).
- Collection : objet enveloppant un tableau (array) avec de nombreuses methodes pratiques (filter, map, sum...).
- Composer : gestionnaire de dependances PHP.
- Controller (Controleur) : classe qui recoit les requetes et orchestre la reponse.
- CORS (Cross-Origin Resource Sharing) : mecanisme de securite du navigateur qui controle quels sites peuvent appeler ton API depuis un domaine different.
- CRUD : Create, Read, Update, Delete — les quatre operations de base sur une donnee.
- CSRF (Cross-Site Request Forgery) : type d attaque ; Laravel s en protege automatiquement via un jeton (token) CSRF cache dans les formulaires.

D

- Dependency Injection (Injection de dependances) : fournir a une classe les objets dont elle a besoin, au lieu qu elle les cree elle-meme.
- Dispatch : action d envoyer un Job dans une file d attente, ou de declencher un Event.
- DTO (Data Transfer Object) : simple objet qui transporte des donnees structurees entre deux couches de l application, sans logique metier.
- DRY (Don t Repeat Yourself) : principe qui consiste a eviter la duplication de code.

E

- Eager Loading : charger a l avance les relations d un Model pour eviter le probleme N+1.
- Eloquent : l ORM (Object-Relational Mapping) integre a Laravel.
- Endpoint : une URL precise exposee par une API, associee a une action.
- Env (.env) : fichier de configuration propre a chaque environnement (dev, prod).
- Event (Evenement) : signal declenche pour notifier qu une action a eu lieu.

F

- Facade : classe qui donne un acces simple et statique a un service du Service Container.
- Factory : classe qui genere de fausses donnees realistes pour les tests.
- Fillable : liste des colonnes autorisees a etre remplies en masse sur un Model.
- Form Request : classe dediee qui regroupe les regles de validation d une requete.
- Framework : ensemble structure d outils et de regles qui accelere le developpement.

G

- Gate : regle d autorisation simple, souvent basee sur une closure.
- Guard : definit comment un utilisateur est authentifie (session, token...).
- Guarded : liste des colonnes INTERDITES lors d une assignation de masse (inverse de fillable).

H

- hasMany / hasOne / belongsTo / belongsToMany : types de relations Eloquent entre Models.
- HTTP (HyperText Transfer Protocol) : protocole de communication utilise entre le navigateur et le serveur.
- Header (en-tete HTTP) : information annexe transmise avec une requete/reponse (ex : type de contenu, jeton d autorisation).

I

- IoC (Inversion of Control) : principe ou le framework prend le controle de la creation des dependances.
- Interface (Contrat) : liste de methodes qu une classe s engage a implementer, sans en imposer le detail.
- Idempotent : une operation qui produit le meme resultat qu on l execute une ou plusieurs fois (ex : DELETE, PUT).

J - K

- Job : classe representant une tache a executer, souvent en arriere-plan via une Queue.
- JSON (JavaScript Object Notation) : format leger d echange de donnees, tres utilise par les API.

- JWT (JSON Web Token) : format de jeton signe utilise pour transmettre une identite de maniere securisee.
- Kernel (Noyau) : chef d orchestre qui gere le cycle de vie d une requete HTTP ou d une commande console.

L - M

- Lazy Loading : chargement d une relation Eloquent SEULEMENT quand on y accede (a l origine du probleme N+1).
- Mailable : classe representant un email a envoyer.
- Mass Assignment : remplissage en masse des attributs d un Model a partir d un tableau de donnees (proteger via fillable/guarded).
- Middleware : filtre qui intercepte une requete avant/apres le controleur.
- Migration : fichier decrivant une modification versionnee de la structure de la base de donnees.
- Model : classe representant une entite metier et sa table en base de donnees.
- Mutator (Mutateur) : methode qui transforme une valeur au moment ou on l ENREGISTRE sur un Model.
- MVC : Model - View - Controller, le patron d architecture principal de Laravel.

N - O

- N+1 (Probleme N+1) : execution d une requete supplementaire pour chaque element d une liste au lieu d une seule requete groupee.
- Namespace (espace de noms) : mecanisme PHP qui evite les conflits de noms entre classes.
- Observer : classe centralisant les reactions aux evenements du cycle de vie d un Model.
- ORM (Object-Relational Mapping) : outil qui transforme les tables en objets manipulables et inversement.

P

- Pagination : decoupage d une longue liste de resultats en plusieurs pages.
- Pivot Table (table pivot) : table intermediaire utilisee pour une relation belongsToMany.
- Policy : classe regroupant les regles d autorisation pour un Model precis.
- Provider (Service Provider) : classe qui enregistre et configure les services de l application au demarrage.

- PSR (PHP Standard Recommendations) : ensemble de standards partages par la communaute PHP (style de code, autoloading...).

Q - R

- Query Builder : outil qui construit des requetes SQL de maniere fluide et chainee, sans passer par un Model Eloquent.
- Queue (file d attente) : mecanisme qui reporte l execution de taches longues en arriere-plan.
- Repository (pattern Repository) : couche qui isole l acces aux donnees derriere une interface.
- REST (Representational State Transfer) : style d architecture pour construire des API.
- Resource (API Resource) : classe qui transforme un Model en structure JSON controlee.
- Route : association entre une URL, une methode HTTP, et une action (controleur ou fonction).
- Route Model Binding : recuperation automatique d un Model a partir d un parametre de route.

S

- Scope (local scope) : methode reutilisable sur un Model pour filtrer une requete Eloquent.
- Seeder : classe qui remplit la base de donnees avec des donnees de depart.
- Service Container : "boite" centrale qui construit et fournit automatiquement les objets (injection de dependances).
- Session : mecanisme de conservation d informations propres a un utilisateur entre plusieurs requetes.
- Singleton : instance UNIQUE d un objet, reutilisee partout ou elle est demandee.
- Soft Delete (suppression douce) : marquer une ligne comme supprimee (colonne deleted_at) sans la supprimer physiquement.
- SQL Injection (injection SQL) : attaque consistant a injecter du code SQL malveillant via une entree utilisateur ; Eloquent et le Query Builder en protegent par defaut grace aux requetes preparees.

T

- Timestamps : colonnes created_at et updated_at gerees automatiquement par Eloquent.

- Tinker : console interactive pour tester du code PHP avec l application chargee.
- Trait : mecanisme PHP permettant de partager des methodes entre plusieurs classes sans heritage direct.
- Transaction : groupe d operations en base de donnees qui reussissent ou echouent ensemble ("tout ou rien").

U - Z

- Unique (regle de validation) : verifie qu une valeur n existe pas deja dans une table.
- Validation : verification des donnees entrantes avant de les traiter ou de les enregistrer.
- View (Vue) : fichier (souvent Blade) charge de l affichage final, sans logique metier complexe.
- Webhook : URL exposee par ton application pour recevoir des notifications automatiques d un service externe (ex : confirmation de paiement Stripe).
- WebSocket : protocole permettant une communication bidirectionnelle et en temps reel entre client et serveur (utilise par le Broadcasting).
- XSS (Cross-Site Scripting) : attaque consistant a injecter du code malveillant (souvent JavaScript) dans une page ; Blade s en protege via l echappement automatique {{ }}.

Chapitre 21 — Problemes courants rencontres en developpement Laravel (et leurs solutions)

Cette section reunit les erreurs et blocages les plus frequemment rencontres, aussi bien par les debutants que par les developpeurs plus experimentes.

Probleme / Erreur	Cause probable et solution
Page blanche ou erreur 500	APP_DEBUG=false cache le detail de l erreur. Passer temporairement APP_DEBUG=true en local, et consulter storage/logs/laravel.log pour le detail exact.
419 PAGE EXPIRED	Le jeton CSRF est absent, invalide ou expire (souvent une session trop vieille). Verifier la presence de @csrf dans le formulaire Blade et que les cookies de session fonctionnent.

Probleme / Erreur	Cause probable et solution
Class not found / Target class does not exist	Probleme d autoloading Composer (mauvais namespace, fichier mal place) ou cache de configuration obsolete. Lancer composer dump-autoload puis php artisan config:clear.
Probleme N+1 (lenteur inexpliquee)	Relations Eloquent chargees en boucle sans eager loading. Utiliser with() pour precharger les relations necessaires.
MassAssignmentException	Tentative de remplir un attribut qui n est pas dans $fillable (ou qui est dans $guarded). Ajouter le champ a $fillable si l usage est legitime.
SQLSTATE[42S02]: Base table not found	Une migration n a pas ete executee. Lancer php artisan migrate.
CORS bloque les appels depuis le frontend	Le domaine du frontend n est pas autorise. Configurer le fichier config/cors.php (ou l equivalent) pour lister les origines autorisees.
Les changements de .env ne sont pas pris en compte	La configuration a ete mise en cache. Lancer php artisan config:clear (et ne jamais utiliser env() en dehors des fichiers config/).
Erreur 401 Unauthenticated sur une API	Le jeton d authentification (Sanctum/Passport) est absent ou invalide dans l en-tete Authorization.
Erreur 403 Forbidden	L utilisateur est authentifie mais n a pas la permission (Gate/Policy) requise pour cette action.
Migration deja executee, colonne deja existante	On tente de relancer une migration qui a deja ete appliquee. Verifier avec php artisan migrate:status ou creer une nouvelle migration pour modifier la table existante.
Uploads de fichiers introuvables publiquement	Le lien symbolique storage n a pas ete cree. Lancer php artisan storage:link.
SQL Injection potentielle	Utilisation de requetes SQL brutes avec concatenation de variables non echappees. Toujours utiliser les requetes preparees d Eloquent/Query Builder, ou les liaisons de parametres (?).

Probleme / Erreur	Cause probable et solution
Fuite de performance en production (queries lentes)	Absence d index sur les colonnes filtrees/jointes frequemment ; absence de cache ; requetes non optimisees. Analyser avec des outils comme Laravel Telescope ou Debugbar.
Erreur de fuseau horaire / dates incoherentes	Mauvaise configuration de timezone dans config/app.php, ou confusion entre l heure du serveur et celle de l utilisateur.

Chapitre 22 — Questions d entretien frequentes (avec reponses)

Ces questions sont classees par niveau de difficulte. L objectif n est pas de reciter mot pour mot, mais de comprendre suffisamment chaque concept pour l expliquer avec tes propres mots, si possible avec un exemple concret.

Niveau debutant

1. Qu est-ce que Laravel et pourquoi l utiliser plutot que du PHP pur ? -> Un framework PHP qui fournit une structure, des outils de securite, un ORM et une organisation de code deja prets, evitant de tout reconstruire depuis zero.
2. Qu est-ce que le patron MVC ? -> Model (donnees), View (affichage), Controller (logique/orchestration) : trois responsabilites separees pour un code plus organise.
3. A quoi sert le fichier .env ? -> A stocker la configuration propre a chaque environnement (identifiants de base de donnees, cles API), sans jamais la partager publiquement.
4. Quelle est la difference entre GET et POST ? -> GET sert a LIRE une donnee (parametres visibles dans l URL), POST sert a ENVOYER/CREER une donnee (corps de requete, souvent invisible dans l URL).
5. Qu est-ce qu Eloquent ? -> L ORM integre a Laravel qui permet de manipuler la base de donnees avec des objets PHP plutot qu en ecrivant du SQL a la main.
6. Comment cree-t-on une nouvelle route ? -> En ajoutant une ligne dans routes/web.php ou routes/api.php avec Route::get/post/put/delete(...).
7. A quoi sert la commande php artisan migrate ? -> A executer les migrations en attente pour mettre a jour la structure de la base de donnees.

Niveau intermediaire

1. Explique le probleme N+1 et comment le resoudre. -> Une requete initiale suivie d une requete supplementaire pour chaque element d une collection au lieu d une seule requete groupee ; on le resout avec l eager loading via la methode with().
2. Quelle est la difference entre fillable et guarded ? -> fillable est une liste blanche (champs autorises a l assignation de masse) ; guarded est une liste noire (champs interdits). Les deux protegent contre le Mass Assignment.
3. Explique la difference entre une Migration et un Seeder. -> La migration definit la STRUCTURE de la base (tables, colonnes) ; le seeder insere des DONNEES dans cette structure.
4. A quoi sert un Middleware ? Donne un exemple. -> Un filtre qui intercepte une requete avant/apres le controleur ; exemple : le middleware auth qui bloque l acces si l utilisateur n est pas connecte.
5. Quelle est la difference entre une relation hasMany et belongsToMany ? -> hasMany relie un parent a plusieurs enfants directement (cle etrangere sur la table enfant) ; belongsToMany relie deux tables via une table pivot, chacune pouvant avoir plusieurs correspondances de l autre cote.
6. Qu est-ce qu une Form Request et pourquoi l utiliser ? -> Une classe dediee a la validation d une requete, qui allege le controleur et centralise les regles de validation et d autorisation.
7. Explique la difference entre authentification et autorisation. -> L authentification verifie l identite (qui es-tu) ; l autorisation verifie les droits (as-tu la permission de faire cette action).
8. Qu est-ce qu une API Resource et a quoi sert-elle ? -> Une classe qui transforme un Model Eloquent en structure JSON precise, permettant de cacher des champs sensibles et de formater les donnees pour l API.
9. Quelle est la difference entre Query Builder et Eloquent ? -> Le Query Builder construit des requetes SQL fluides sans notion de Model (retourne des objets generiques) ; Eloquent est construit au-dessus et retourne des instances de Model avec toute leur logique metier.

Niveau avance

1. Explique le fonctionnement du Service Container (IoC) de Laravel. -> C est le mecanisme central qui construit et fournit automatiquement les objets dont une classe a besoin (injection de dependances), en resolvant les interfaces vers leurs implementations concretes enregistrees via bind() ou singleton().

2. Quelle est la difference entre bind() et singleton() dans le Service Container ? -> bind() cree une nouvelle instance a chaque resolution ; singleton() reutilise toujours la meme instance pendant la duree de vie de l application/requete.
3. Facade vs Injection de dependances : avantages et inconvenients ? -> Les Facades sont plus courtes et lisibles mais masquent la dependance reelle de la classe, ce qui peut compliquer les tests unitaires ; l injection explicite via le constructeur rend les dependances visibles et facilite le remplacement par des mocks en test.
4. Comment fonctionnent les Jobs et les Queues, et pourquoi les utiliser ? -> Un Job encapsule une tache potentiellement longue ; on la "dispatch" dans une file d attente au lieu de l executer immediatement, pour repondre plus vite a l utilisateur ; un worker (queue:work) consomme ensuite la file en arriere-plan.
5. Comment proteger une application Laravel contre les failles de securite courantes ? -> Utiliser fillable/guarded contre le mass assignment, les requetes preparees d Eloquent contre l injection SQL, l echappement Blade {{}} contre le XSS, le jeton CSRF sur les formulaires, le rate limiting contre les abus d API, et ne jamais exposer APP_DEBUG=true en production.
6. Quelle est la difference entre une Policy et une Gate ? -> Une Gate est une regle d autorisation generale, souvent une simple closure ; une Policy regroupe TOUTES les regles d autorisation liees a UN Model precis, de maniere organisee.
7. Comment optimiserais-tu les performances d une application Laravel en production ? -> Mettre en cache la configuration/les routes (config:cache, route:cache), utiliser l eager loading pour eviter le N+1, indexer les colonnes filtrees en base, utiliser un cache applicatif (Redis) pour les donnees couteuses a calculer, et deporter les taches lourdes vers des Queues.
8. Explique le cycle de vie complet d une requete Laravel. -> (voir Chapitre 2) De public/index.php au Kernel HTTP, en passant par les middlewares, le routeur, le controleur, le Model, jusqu a la reponse renvoyee au client.
9. Qu est-ce qu un design pattern Repository et pourquoi certaines equipes l utilisent-elles avec Laravel malgre Eloquent ? -> Une couche d abstraction entre les controleurs et Eloquent, souvent derriere une interface, qui permet de changer la source de donnees (base SQL, API externe, cache) sans modifier le code appelant, et facilite les tests en isolant l acces aux donnees.

Chapitre 23 — Conseils pratiques pour se preparer a un entretien Laravel / backend

1. Ne memorise jamais un terme sans etre capable de l expliquer avec une analogie simple (comme celles de ce document) : c est le signe que tu as VRAIMENT compris, pas seulement retenu un mot.

2. Construis un petit projet complet de A a Z (par exemple une gestion de taches ou un mini e-commerce) qui utilise : migrations, relations Eloquent, authentification, validation, une API et des tests. La pratique consolide bien plus que la seule lecture.
3. Entraine-toi a expliquer a voix haute le probleme N+1 et le Service Container : ce sont deux des questions les plus posees et les plus discriminantes en entretien.
4. Familiarise-toi avec la commande php artisan route:list et php artisan tinker : elles sont tres utiles pour explorer rapidement un projet Laravel que tu decouvres, y compris en entretien technique live.
5. Prepare 2 ou 3 exemples concrets de "bug resolu" ou de "probleme de performance corrige" issus de tes propres projets : les recruteurs adorent les histoires concretes plutot que la theorie seule.
6. Lis regulierement le code source d un package Laravel populaire (par exemple un package d authentification) pour voir comment les concepts de ce document (Service Provider, Facade, Middleware) sont utilises en conditions reelles.

Chapitre 24 — Pour aller plus loin

- Documentation officielle Laravel : laravel.com/docs (a lire en priorite, toujours a jour et tres claire).
- Laracasts : plateforme de tutoriels video specialement dediee a Laravel et a l ecosysteme PHP.
- Laravel News : actualites et bonnes pratiques de l ecosysteme.
- Le code source de Laravel lui-meme sur GitHub (organisation "laravel") : tres instructif pour voir les patterns de conception appliques en conditions reelles.

Ce document couvre l essentiel du jargon et des mecanismes du backend Laravel. La maitrise complete viendra avec la pratique repetee : construire, casser, deboguer, recommencer.